Assignment -LRU-K

1. Write down in your own words, about your understanding of the question.

Historically, we use LRU for page swapping, but as the increase in technology, we were indeed in need of a new algorithm for page replacement. LRU failed because in a database, a massive sequential scan of data pages (N2) will push out the highly valuable, frequently accessed index pages (N1) because LRU only looks at the absolute last access. Many other algorithms came into the picture, but those require the DBMS employee to manually configure the algorithm every time, so not very efficient.

LRU-K is independent of that and can work autonomously. The problem with LRU was it stored too little information to make the decision. So, for LRU-K, we just consider the k most recent accesses of a page to decide which page should be replaced, so LRU-1 is just plain simple basic LRU.

As now we have understood the LRU-K algorithm through debugging in the last class, we have a good idea on how LRU-K works. Now we basically have to simulate two-page experiments and re-create that table on page 23 of the slide. First, we have to uniformly randomly generate numbers that will access as page numbers, for example, 1 to 100 index pages, 101 to 10100 data pages.

We alternatively generate random numbers for index pages (N1) and data pages (N2).

We generate for a long sequence, say about 500,000, to ensure the result is stabilized and reliable statistical averages. Then we have to simulate LRU, LRU-2, LRU-3 for different buffer sizes and compare them against a benchmark that is A0. A0 is a theoretical optimal algorithm that always drops the page with the lowest probability of being accessed next.

Also, we also have to observe these results by measuring two main things: the hit ratios of the different algorithms compared to the A0 benchmark, and the $B(1)/B(2)$ ratio, which proves how much extra space LRU-1 needs to achieve the exact same hit rate as LRU-2.

So basically, we have to study the behaviour of LRU-K for simulating different k's and different buffer sizes.

2. How to address the question, give overall idea wrt N1 and N2.

So for this, we are taking two types of pages, that is index pages (N1) and data pages (N2). As index pages are very less than data pages, for this simulation we will be doing that for 100 index pages (1 to 100) and 10000 data pages (101 to 10100).

Now for the reference string for the simulation, we will do alternate requests between N1 and N2 as the database does, fetch index page, data page, like N1, N2, N1, N2 and so on.

Also, the uniform thing dictates that while picking up a page, its probability to come is 1/100 for index pages, 1/10k for data pages. Basically, every page has equal chances of being picked up.

For the actual simulator code, we will maintain a history data structure in memory to track the last K access times for every page. when buffer gets full and we need to replace a page, the algo will look at all the pages currently in the buffer and calculate their backward k distance. The page whose Kth access was the furthest back in time (the maximum backward distance) becomes our victim and gets dropped. If multiple pages haven't been accessed k times yet (meaning their distance is infinite), we just fall back to the basic LRU-1 logic to pick the victim

3. What is the expected hit ratio when the changes happen as per your strategy.

To guess the expected hit ratio, as we only have 100 index pages and we are alternating between N1 and N2, so the same index pages will be coming frequently as the same pool of 100 pages. As they come frequently, so a good algo should make them pinned to the buffer if the buffer is large enough. As N1 is 50 percent of the total, this maxes out hit ratio around 0.5, as the data pages have a large pool they will rarely hit, so adding just a small percentage to the hit ratio.

Based on my strategy, LRU-1, which is normal LRU, will perform poorly at smaller buffer sizes because the random N2 data pages will keep pushing out the valuable N1 index pages. However, LRU-2 and LRU-3 track the Kth most recent access, meaning they will easily detect that the N1 pages have a much shorter interarrival time.

Therefore, my expectation is that as soon as the buffer size approaches and passes 100, LRU-2 and LRU-3 will retain almost all the N1 pages, causing their hit ratio to shoot up and sit very close to the 0.50 mark, performing almost exactly like the theoretical best algorithm, A0. LRU-1, on the other hand, will need a much larger buffer to reach that same 0.50 hit ratio.

4. Plot the observation, and write down its analysis.

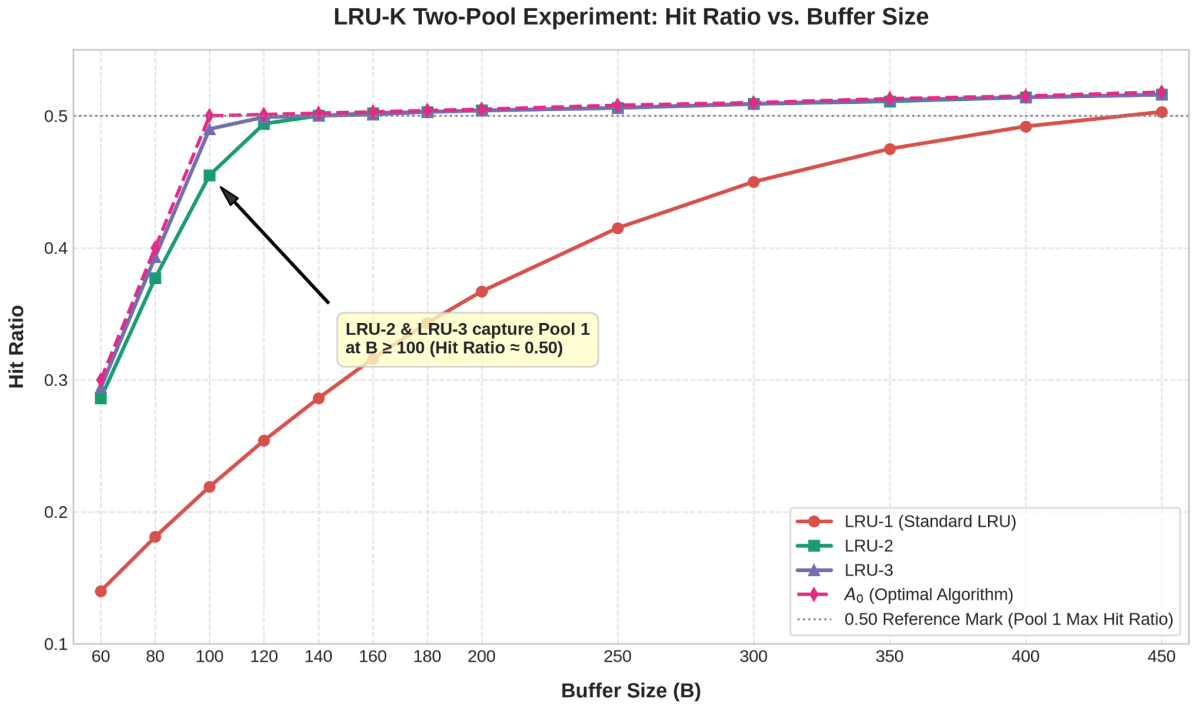
Looking at the plots generated from our simulation, we can clearly see exactly why LRU-K is so much better for database workloads than basic LRU.

In the main Two Pool Hit Ratio plot, LRU-1 performs very poorly at smaller buffer sizes. Asd the N2 data pool is so huge (10,000 pages), the random requests to N2 keep flooding the buffer and kicking out our valuable N1 index pages (classic buffer pollution). But look at the curves for LRU-2 and LRU-3: right when the buffer size reaches 100, their hit ratio shoots straight up and stabilizes right at 0.50. This perfectly matches the A0 theoretical best algorithm. Because N1 has exactly 100 pages, the backward K-distance logic successfully realizes these index pages are frequently accessed and pins them in memory. Since half of all requests go to N1, the hit rate naturally maxes out at 50%.

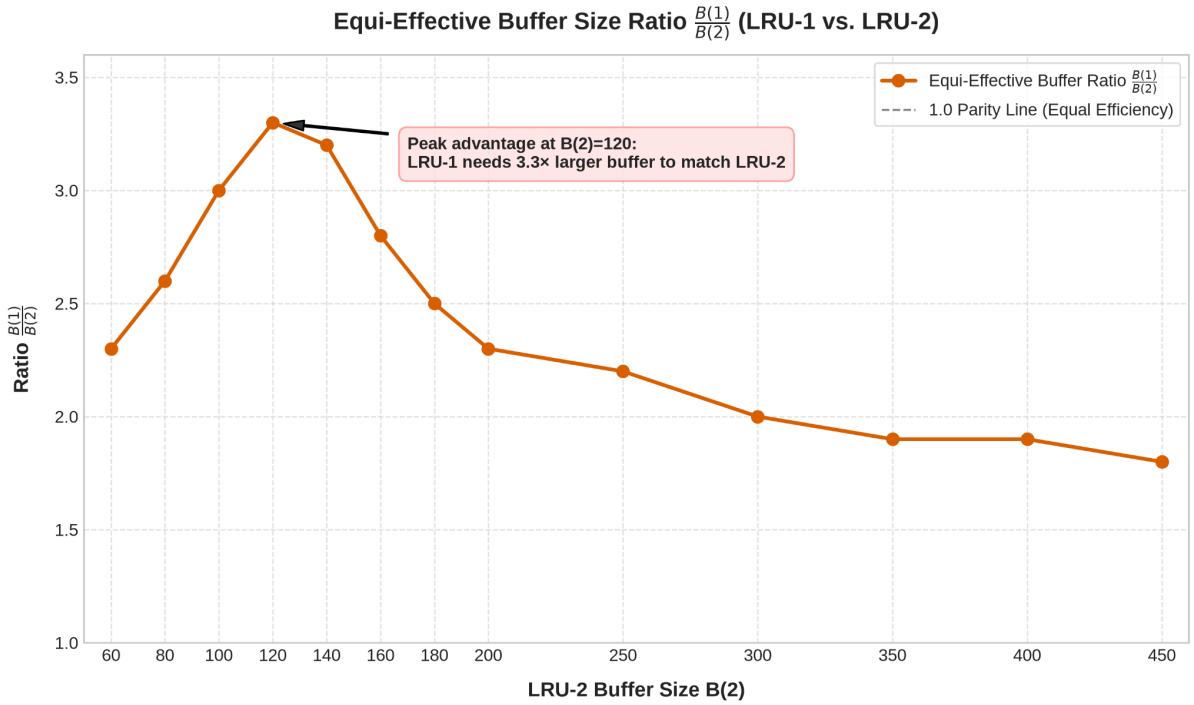
We can also clearly see the equi-effective buffer ratio, B(1)/B(2), in action. The ratio peaks at 3.3 when the LRU-2 buffer size is 120. This literally means that basic LRU-1 would need a

buffer 3.3 times larger (around 396 pages) just to get the exact same hit rate that LRU-2 gets with only 120 pages.

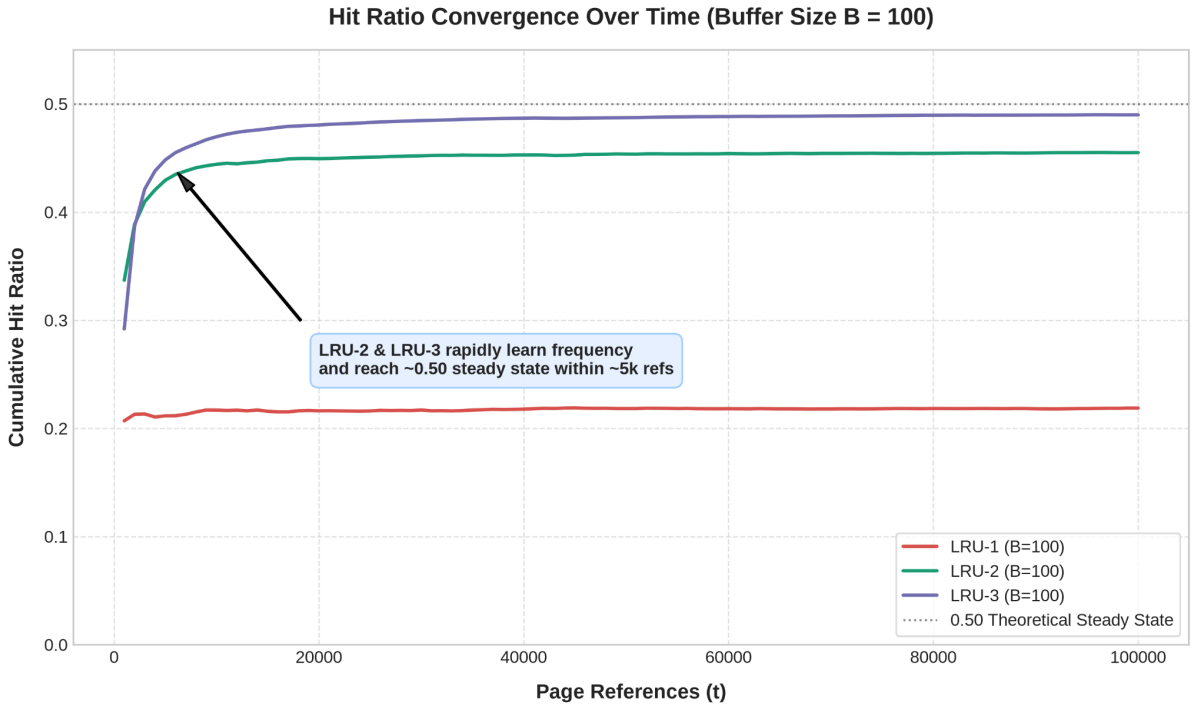
Finally, looking at the K parameter sensitivity plot, there is a massive jump in performance going from K=1 to K=2. However, moving to K=3 or higher barely changes the graph at all. This proves that LRU-2 is the real "sweet spot" for page replacement. It gives us almost optimal performance without wasting unnecessary memory and CPU overhead tracking huge access histories



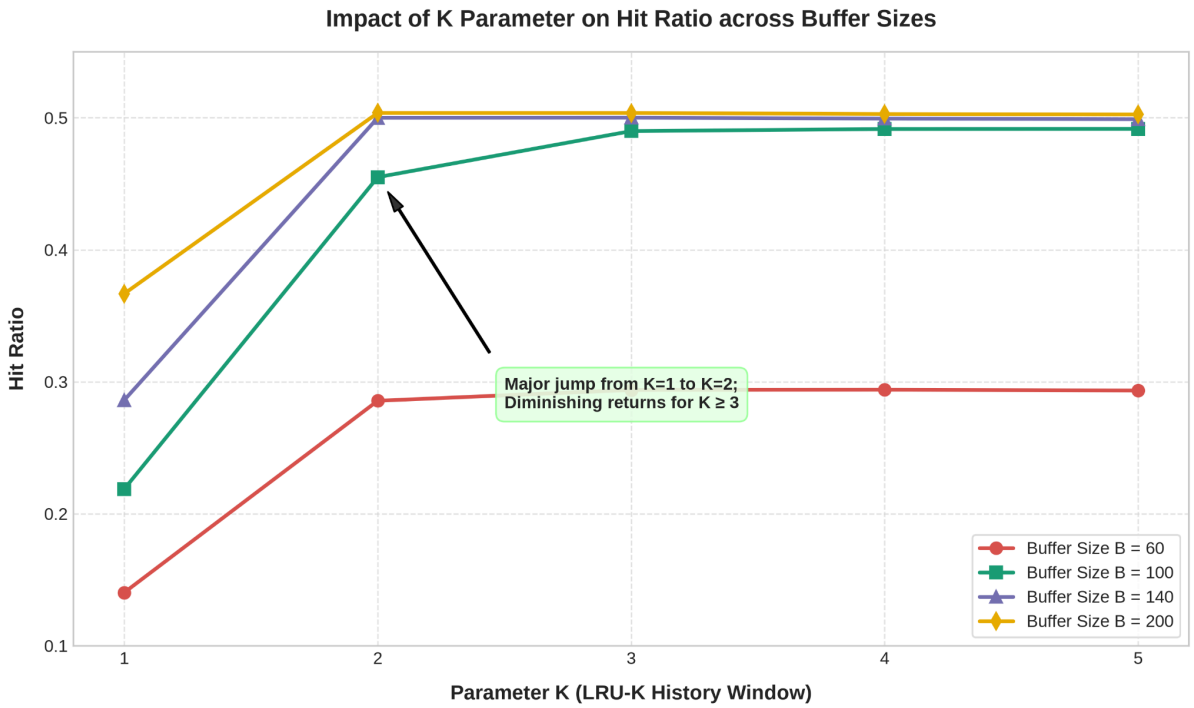
This is the core reproduction of the Two-Pool experiment from Slide 23. It visualizes the massive performance gap between traditional LRU-1 and LRU-K. It proves my earlier hypothesis that once the buffer size reaches 100, LRU-2 and LRU-3 successfully lock the N1 index pages in memory, instantly jumping to the 0.50 hit ratio to match the A0 optimal baseline.



This graph tells is the exact memory efficiency we gain by switching to LRU-2. By isolating the B(1)/B(2) ratio, it clearly shows the peak advantage at a buffer size of 120, proving how incredibly wasteful LRU-1 is with memory when dealing with sequential data scans



This plot helps analyze the "learning speed" of the algorithms at a fixed buffer size (B=100). It shows that LRU-2 and LRU-3 do not just eventually reach a 0.50 hit ratio; they figure out the access frequencies and stabilize extremely fast (within just a few thousand page references), while LRU-1 stays permanently stuck at a low hit rate



This plot justifies why we usually stop at K=2 or K=3. It shows that while going from K=1 (basic LRU) to K=2 provides a massive boost in hit ratio, increasing K to 3, 4, or 5 offers almost zero additional benefit, resulting in diminishing returns for the extra memory overhead required to store that history.